

AW: Code-Anfrage zu "Towards Data-Based Cache Optimization of B+-Trees" (DaMoN 2023)

Probst, Benjamin-Elias

Fr 08.05.2026 17:48

Gesendete Elemente

An: Roland Kühn <roland.kuehn@tu-dortmund.de>;

Cc: Habich, Dirk <dirk.habich@tu-dresden.de>;

Hallo Roland,

In Beziehung der CacheEngine sehe ich es als meine Aufgabe an, den bestehenden Code so gut wie perfekt unangetastet zu lassen, einzelne Modulklassen über multiple Suchalgorithmen architektonisch zu kategorisieren, und final alle Permutationen aller bestehenden Algorithmen-Konzepte als Einzelexperiment mit und ohne bewusste Cache Informationen zu testen. Die Kernfrage besteht dabei darin, ob die CacheEngine als Modell der zur Verfügung gestellten CPU ein Framework bereitstellen kann, welches ein Such-Algorithmus-Konzept zur eigenen Optimierung verwenden kann. Die CacheEngine an sich ist also neu, der Code vergangener Experimente wird wiederverwendet. Im Prinzip baue ich also ein Framework zu bewerten und Testen von Suchalgorithmen mit dem Vergleich zwischen heuristischen und informierten Architektur-Entscheidungen und automatischer Anpassung.

Plattform zum Testen ist derzeit ein K8s Cluster auf Talos OS mit einer Produktion auf einem Ryzen 9 9950X3D und einem i9 14900KS, je mit 64GB RAM bei 5600 MIT DDR5 RAM und einer Latenz von 36 Zyklen.

Ich hätte bitte gerne den Code, das ist natürlich ein eher seltener Fall, das der Code nur experimentell ist, aber ich werde mir die Mühe machen ihn zu Ordnen und zu zerlegen. Bitte einen Link schicken, mein Postfach ist voll, ich hab noch 15MB freien Speicher, damit ich den Code herunterladen kann.

Mit freundlichen Grüßen,
Benjamin-Elias Probst

Von: Roland Kühn <roland.kuehn@tu-dortmund.de>

Gesendet: Freitag, 8. Mai 2026 10:35:47

An: Probst, Benjamin-Elias

Betreff: Re: Code-Anfrage zu "Towards Data-Based Cache Optimization of B+-Trees" (DaMoN 2023)

Hallo Benjamin-Elias,

ich hoffe, es ist okay, wenn wir uns duzen :). Das Thema deiner Diplomarbeit klingt auf jeden Fall spannend und passt thematisch sehr gut zu den offenen Fragen aus unserem Paper. Wir hatten tatsächlich auch einmal eine Studienarbeit, die sich mit Runtime-Reordering im Adaptive Radix Tree (ART) beschäftigt hat. Damals sind wir allerdings nicht zu besonders überzeugenden Ergebnissen gekommen und haben das Thema anschließend nicht weiter mit großem Nachdruck verfolgt. Umso gespannter bin ich jetzt auf deine Ergebnisse. Ich würde mich freuen, wenn du mich ein wenig auf dem Laufenden hältst, sofern dir das möglich ist.

Was den Code aus dem Paper betrifft, muss ich allerdings eine Sache klarstellen: Im Paper ging es uns in erster Linie darum, die eigentlichen Effekte des Reorderings sichtbar zu machen. Für die Evaluation war es uns daher wichtig, eine möglichst starke und unverfälschte Baseline zu haben. Wir wollten vermeiden, die Baseline allein durch zusätzliches Sammeln von Zugriffsinformationen zu verschlechtern und uns dadurch einen unfairen Vorteil für das Reordering zu verschaffen. Entsprechend haben wir uns dort bewusst nicht näher mit der Frage beschäftigt, wie sich die Histogrammbildung bzw. die Erfassung der Zugriffsinformationen möglichst effizient zur Laufzeit umsetzen lässt.

Wir hatten damals zwar auch naive Varianten ausprobiert, bei denen die Zugriffe direkt während der Traversierung in jedem Knoten mitgetrackt wurden. Gerade bei der parallelen Ausführung führte das jedoch recht schnell zu deutlichen Performance-Einbußen. Insbesondere auf die Knoten in den oberen Ebenen des Baums wird sehr häufig zugegriffen, sodass diese typischerweise in den Caches mehrerer Kerne liegen. Zusätzliche Schreibzugriffe führen dann durch die Cache-Kohärenzmechanismen schnell zu starkem Cacheline-Ping-Pong zwischen den Kernen, was die Performance entsprechend beeinträchtigt. Die Zugriffshäufigkeiten wurden für die Evaluation daher offline rekonstruiert: Wir wussten, welche Keys wie häufig gelesen wurden, sind anschließend die jeweiligen Pfade im Baum entlanggelaufen, haben die Zugriffszahlen der Knoten aufaddiert und den Baum danach entsprechend umsortiert.

Das Ganze ist natürlich schwierig, wenn man keine Informationen darüber hat, wie häufig auf welche Keys zugegriffen wird. Wir haben daher mittlerweile auch eine vergleichsweise einfache Variante implementiert, bei der die Zugriffsinformationen nicht in allen Knoten, sondern nur in den Blatt-Knoten gespeichert werden. Vor dem eigentlichen Reordering werden dann alle Blätter einmal bis zur Wurzel traversiert, wobei die Zugriffszahlen für die inneren Knoten aufsummiert werden. Dadurch muss während der regulären Baum-Traversierung nur ein einzelner Counter im Blatt-Knoten aktualisiert werden. Gleichzeitig ist in den unteren Ebenen des Baums die Wahrscheinlichkeit geringer, dass viele Threads denselben Blatt-Knoten bearbeiten, wodurch der Overhead durch Cache-Kohärenz kleiner ausfällt. Wir haben das Ganze zusätzlich um eine Sampling-basierte Variante erweitert, bei der nicht jeder Zugriff, sondern nur jeder n-te Zugriff im Blatt-Knoten gezählt wird. Dadurch lässt sich der Runtime-Overhead noch einmal reduzieren und man erhält einen konfigurierbaren Trade-off zwischen Genauigkeit und Overhead.

Ich denke, dass wir euch den Code grundsätzlich zukommen lassen können. Ich muss allerdings dazu sagen, dass es sich dabei eher um experimentellen Evaluationscode als um ein sauber gekapseltes Framework handelt. Ich müsste daher zumindest noch einmal den Code durchgehen und alles ein wenig aufräumen und dokumentieren ;)

Beste Grüße nach Dresden

Roland

On 07.05.26 02:45, Benjamin-Elias Probst wrote:

Sehr geehrter Herr Kuehn,

ich arbeite derzeit an meiner Diplomarbeit an der TU Dresden unter Prof. Dr. Dirk Habich ("Active Cache-Aware Hardware Adaptation Cache Engine for Trie-Based Index Structures"). Eine der vier zentralen Forschungsluecken adressiert genau das, was Sie in DaMoN 2023 als bewusst-aussen-gelassen genannt haben:
Online Re-Layouting ueber Histogramm-basierte Hot-Path-Detection.

Ihr Histogramm-pro-Knoten-Mechanismus (Sec.3) ist eine direkte Inspiration fuer die PRT-ART Cache-Engine. Daher waere es sinnvoll, wenn ich mir bitte den Originalcode (mit den Histogramm-Strukturen + B+-Tree-Integration) fuer die Adapter-Konstruktion und Vergleichsmessungen etwas genauer ansehen koennte.

Frage: Ist Ihr DaMoN-2023-Code fuer akademische Diplomarbeit-Zwecke verfuegbar?
Welcher Lizenztyp liegt auf derzeit auf dem Code?

Falls eine Code-Freigabe schwierig ist, wuerde mich auch ein Pseudocode-Auszug aus Ihren internen Notizen fuer eine sauberer Re-Implementation reichen.

Mit freundlichen Gruessen,
Benjamin-Elias Probst (s2631336@tu-dresden.de)

--

Roland Kühn
TU Dortmund University, DBIS Group
Otto-Hahn-Strasse 14, 44227 Dortmund, Germany